

NAME : T.Jaya Krishna Reddy

REG NO : 192572036

COURSE : AI

1 Write the python program to solve 8-Puzzle problem

Program code :

```
from collections import deque
```

```
def get_moves(state):
```

```
    moves = []
```

```
    zero = state.index(0)
```

```
    row = zero // 3
```

```
    col = zero % 3
```

```
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
    for dr, dc in directions:
```

```
        nr = row + dr
```

```
        nc = col + dc
```

```
        if 0 <= nr < 3 and 0 <= nc < 3:
```

```
            new_zero = nr * 3 + nc
```

```
            new_state = list(state)
```

```
new_state[zero], new_state[new_zero] = \
    new_state[new_zero], new_state[zero]

moves.append(tuple(new_state))

return moves
```

```
def solve(start, goal):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path

        for new_state in get_moves(state):
            if new_state not in visited:
                visited.add(new_state)
```

```
queue.append((new_state, path + [new_state]))
```

```
return None
```

```
start = (1, 2, 3,  
         4, 0, 6,  
         7, 5, 8)
```

```
goal = (1, 2, 3,  
        4, 5, 6,  
        7, 8, 0)
```

```
solution = solve(start, goal)
```

```
print("8-Puzzle Solution:")
```

```
for state in solution:
```

```
    print()
```

```
    for i in range(0, 9, 3):
```

```
        print(state[i], state[i+1], state[i+2])
```

Out Put :

8-Puzzle Solution:

1 2 3

4 0 6

7 5 8

1 2 3

4 5 6

7 0 8

1 2 3

4 5 6

7 8 0

2 Write the python program to solve 8-Queen problem

Program Code :

```
def is_safe(board, row, col):
```

```
    for i in range(row):
```

```
        if board[i] == col:
```

```
            return False
```

```
    if abs(board[i] - col) == abs(i - row):  
        return False
```

```
    return True
```

```
def solve_queens(board, row):
```

```
    if row == 8:
```

```
        return True
```

```
    for col in range(8):
```

```
        if is_safe(board, row, col):
```

```
            board[row] = col
```

```
            if solve_queens(board, row + 1):
```

```
                return True
```

```
            board[row] = -1
```

```
    return False
```

```
board = [-1] * 8
```

```
if solve_queens(board, 0):  
    print("8-Queen Solution:")
```

```
    for row in range(8):  
        for col in range(8):  
            if board[row] == col:  
                print("Q", end=" ")  
            else:  
                print(".", end=" ")  
        print()
```

Output :

8-Queen Solution:

Q

. . . . Q . . .

. Q

. Q . .

. . Q

. Q .

. Q

... Q ...

3. Python Program for Water Jug Problem

Program Code :

```
from collections import deque
```

```
def water_jug():
```

```
    visited = set()
```

```
    queue = deque()
```

```
    start = (0, 0)
```

```
    queue.append((start, []))
```

```
    while queue:
```

```
        (a, b), path = queue.popleft()
```

```
        if (a, b) in visited:
```

```
            continue
```

```
        visited.add((a, b))
```

```
        path = path + [(a, b)]
```

```
        if a == 2:
```

```
            return path
```

```
states = [  
    (4, b),    # Fill 4L jug  
    (a, 3),    # Fill 3L jug  
    (0, b),    # Empty 4L jug  
    (a, 0),    # Empty 3L jug  
    (a - min(a, 3-b), b + min(a, 3-b)),  
    (a + min(b, 4-a), b - min(b, 4-a))  
]
```

```
for state in states:  
    if state not in visited:  
        queue.append((state, path))
```

```
solution = water_jug()
```

```
print("Water Jug Solution:")
```

```
for state in solution:  
    print("4L Jug =", state[0], " | 3L Jug =", state[1])
```

Output :

Water Jug Solution:

4L Jug = 0 | 3L Jug = 0

4L Jug = 0 | 3L Jug = 3

4L Jug = 3 | 3L Jug = 0

4L Jug = 3 | 3L Jug = 3

4L Jug = 4 | 3L Jug = 2

4L Jug = 0 | 3L Jug = 2

4L Jug = 2 | 3L Jug = 0

4. Python Program for Crypt-Arithmetic Problem

Program Output :

```
from itertools import permutations
```

```
letters = "SENDMORY"
```

```
for values in permutations(range(10), 8):
```

```
    S, E, N, D, M, O, R, Y = values
```

```
    if S == 0 or M == 0:
```

```
        continue
```

$$\text{SEND} = 1000*S + 100*E + 10*N + D$$

$$\text{MORE} = 1000*M + 100*O + 10*R + E$$

$$\text{MONEY} = 10000*M + 1000*O + 100*N + 10*E + Y$$

if SEND + MORE == MONEY:

 print("Solution:")

 print("S =", S)

 print("E =", E)

 print("N =", N)

 print("D =", D)

 print("M =", M)

 print("O =", O)

 print("R =", R)

 print("Y =", Y)

print()

print(SEND, "+", MORE, "=", MONEY)

break

Out Put :

Solution:

$$S = 9$$

$$E = 5$$

$$N = 6$$

$$D = 7$$

$$M = 1$$

$$O = 0$$

$$R = 8$$

$$Y = 2$$

$$9567 + 1085 = 10652$$

5. Python Program for Missionaries and Cannibals
from collections import deque

```
def safe(m, c):
```

```
    if m < 0 or c < 0 or m > 3 or c > 3:
```

```
        return False
```

```
    if m > 0 and m < c:
```

```
        return False
```

$mr = 3 - m$

$cr = 3 - c$

if $mr > 0$ and $mr < cr$:

 return False

return True

def solve():

 start = (3, 3, 0)

 goal = (0, 0, 1)

 queue = deque([(start, [start])])

 visited = {start}

 moves = [

 (1, 0), (2, 0),

 (0, 1), (0, 2),

 (1, 1)

]

```
while queue:
```

```
    state, path = queue.popleft()
```

```
    if state == goal:
```

```
        return path
```

```
    m, c, boat = state
```

```
    for dm, dc in moves:
```

```
        if boat == 0:
```

```
            new_state = (m-dm, c-dc, 1)
```

```
        else:
```

```
            new_state = (m+dm, c+dc, 0)
```

```
        if safe(new_state[0], new_state[1]):
```

```
            if new_state not in visited:
```

```
                visited.add(new_state)
```

```
                queue.append((new_state, path + [new_state]))
```

```
return None
```

```
solution = solve()
```

```
print("Missionaries and Cannibals Solution:")
```

```
for state in solution:
```

```
    print(state)
```

Out put:

Missionaries and Cannibals Solution:

(3, 3, 0)

(3, 1, 1)

(3, 2, 0)

(3, 0, 1)

(3, 1, 0)

(1, 1, 1)

(2, 2, 0)

(0, 2, 1)

(0, 3, 0)

(0, 1, 1)

(0, 2, 0)

(0, 0, 1)

6. Python Program for Vacuum Cleaner Problem

Program Code :

```
def vacuum_cleaner():
```

```
    room = {  
        "A": "Dirty",  
        "B": "Dirty"  
    }
```

```
    position = "A"
```

```
    print("Initial State:")
```

```
    print("Room A:", room["A"])
```

```
    print("Room B:", room["B"])
```

```
    while room["A"] == "Dirty" or room["B"] == "Dirty":
```

```
        if room[position] == "Dirty":
```

```
            print("Vacuum cleaning Room", position)
```

```
            room[position] = "Clean"
```

```
        else:
```

```
if position == "A":  
    position = "B"  
    print("Move to Room B")  
else:  
    position = "A"  
    print("Move to Room A")
```

```
print()  
print("Final State:")  
print("Room A:", room["A"])  
print("Room B:", room["B"])
```

```
vacuum_cleaner()
```

Output :

Initial State:

Room A: Dirty

Room B: Dirty

Vacuum cleaning Room A

Move to Room B

Vacuum cleaning Room B

Final State:

Room A: Clean

Room B: Clean

7. Python Program to Implement BFS

Program Code :

```
from collections import deque
```

```
def bfs(graph, start):
```

```
    visited = set()
```

```
    queue = deque([start])
```

```
    while queue:
```

```
        node = queue.popleft()
```

```
        if node not in visited:
```

```
            print(node, end=" ")
```

```
            visited.add(node)
```

```
    for neighbor in graph[node]:  
        if neighbor not in visited:  
            queue.append(neighbor)
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}
```

```
print("BFS Traversal:")  
bfs(graph, 'A')
```

Out put:

BFS Traversal:

A B C D E F

8. Python Program to Implement DFS

Program Code :

```
def dfs(graph, node, visited):

    if node not in visited:

        print(node, end=" ")

        visited.add(node)

    for neighbor in graph[node]:

        dfs(graph, neighbor, visited)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = set()
```

```
print("DFS Traversal:")  
dfs(graph, 'A', visited)
```

Output :

DFS Traversal:

A B D E C F

9. Python Program to Implement Travelling Salesman Problem

Program Code :

```
from itertools import permutations
```

```
cities = ['A', 'B', 'C', 'D']
```

```
distance = {  
    'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},  
    'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},  
    'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},  
    'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}  
}
```

```
start = 'A'
```

```
min_distance = float('inf')
```

```
best_path = None
```

```
for path in permutations([city for city in cities if city != start]):
```

```
    current_path = (start,) + path + (start,)
```

```
    total_distance = 0
```

```
    for i in range(len(current_path) - 1):
```

```
        total_distance +=  
distance[current_path[i]][current_path[i+1]]
```

```
    if total_distance < min_distance:
```

```
        min_distance = total_distance
```

```
        best_path = current_path
```

```
print("Best Path:")
```

```
print(" -> ".join(best_path))
```

```
print("Minimum Distance:", min_distance)
```

Output :

Best Path:

A -> B -> D -> C -> A

Minimum Distance: 80

10. Python Program to Implement A* Algorithm

Program Code :

```
import heapq
```

```
def a_star(graph, heuristic, start, goal):
```

```
    open_list = []
```

```
    heapq.heappush(open_list, (heuristic[start], 0, start, [start]))
```

```
    visited = set()
```

```
    while open_list:
```

```
        f, g, node, path = heapq.heappop(open_list)
```

```
        if node == goal:
```

```
    return path, g

    if node in visited:
        continue

    visited.add(node)

    for neighbor, cost in graph[node]:

        if neighbor not in visited:

            new_g = g + cost
            new_f = new_g + heuristic[neighbor]

            heapq.heappush(
                open_list,
                (new_f, new_g, neighbor, path + [neighbor])
            )

    return None, None
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('A', 1), ('C', 2), ('D', 5)],  
    'C': [('A', 4), ('B', 2), ('D', 1)],  
    'D': [('B', 5), ('C', 1)]  
}
```

```
heuristic = {  
    'A': 4,  
    'B': 3,  
    'C': 1,  
    'D': 0  
}
```

```
start = 'A'
```

```
goal = 'D'
```

```
path, cost = a_star(graph, heuristic, start, goal)
```

```
print("A* Algorithm")
```

```
print("Path:", " -> ".join(path))print("Cost:", cost)
```

Output :

A* Algorithm

Path: A -> B -> C -> D

Cost: 4

OUTPUTS :

OUTPUT 1:

```
>>> = RESTART: C:/Users/T.Jaya Krishna Re
      /program 1.py
      8-Puzzle Solution:

      1 2 3
      4 0 6
      7 5 8

      1 2 3
      4 5 6
      7 0 8

      1 2 3
      4 5 6
      7 8 0
>>> |
```

OUT PUT 2 :

```
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/
/program 1.py
8-Queen Solution:
Q . . . . .
. . . . Q . . .
. . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . .
. . . Q . . . .
>> |
```

OUT PUT 3:

```
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python31
/program 1.py
Water Jug Solution:
4L Jug = 0   | 3L Jug = 0
4L Jug = 4   | 3L Jug = 0
4L Jug = 1   | 3L Jug = 3
4L Jug = 1   | 3L Jug = 0
4L Jug = 0   | 3L Jug = 1
4L Jug = 4   | 3L Jug = 1
4L Jug = 2   | 3L Jug = 3
>> |
```

OUT PUT 4 :

```
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
Solution:
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

9567 + 1085 = 10652
>|
```

OUTPUT 5 :

```
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
Missionaries and Cannibals Solution:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
>>|
```

OUTPUT 6:

```
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
Initial State:
Room A: Dirty
Room B: Dirty
Vacuum cleaning Room A
Move to Room B
Vacuum cleaning Room B

Final State:
Room A: Clean
Room B: Clean
>>
```

OUTPUT 7 :

```
>>> = RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
    /program 1.py
    BFS Traversal:
    A B C D E F
>>>
>>>
>>>
```

OUTPUT 8 :

```
>
>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
DFS Traversal:
A B D E C F
>
Ln: 78 Col:
Ln: 23 Col:
```

OUTPUT 9:

```
A B D E C F
>>>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
Best Path:
A -> B -> D -> C -> A
Minimum Distance: 80
>>>
>>>
>>>
>>>
>>>
>>>
```

OUTPUT 10:

```
>
= RESTART: C:/Users/T.Jaya Krishna Reddy/AppData/Local/Programs/Python/Python314
/program 1.py
A* Algorithm
Path: A -> B -> C -> D
Cost: 4
>
>
>
>
Ln: 97 Col:
```